

Adora — Piecewise-Linear Formal Solver

As-built notes · Ivan Milic · 2026-09-20 · session summary

Upgraded the polarised formal solver from piecewise-constant (DELO) to **piecewise-linear** (DELO-linear). New code lives in `formal_solvers/`; the old `vector_formal_solver.py` is untouched. `delo_linear_fs` is a **drop-in** for `delo_constant_fs` (same signature (`dz`, `I_start`, `emis`, `opac`)) — only the physics changed — and is now wired into `vector_response_fn.py` (see §4).

1. The physics change

The DELO machinery (`stokes_K`, $K' = K/K_{00} - I$, Φ_k , Φ_{k+1} , `jnp.linalg.solve`, `fori_loop`) is unchanged. Only the two source/coupling weights change: the constant $0.5 F_k$ trapezoid becomes the linear-source weights (Olson & Kunasz 1987). With $u = \Delta\tau$, $E = e^{-u}$, $w_0 = 1 - E$, $w_1 = 1 - (1 + u)E$:

$$\psi_a = \frac{w_1}{u} \text{ (upwind: } K'_m, \eta_m), \quad \psi_b = w_0 - \frac{w_1}{u} \text{ (local: } K', \eta), \quad \psi_a + \psi_b = w_0.$$

The sum is unchanged (constant-source limit identical), but the weights redistribute correctly toward the local point as a layer becomes optically thick — which is where the constant scheme's staircase error lives.

2. The gradient hazard (the reason to be careful)

$\psi = w_1/u$ is $0/0$ as $u \rightarrow 0$: the *value* is finite but its **autodiff derivative is NaN**, and small u is exactly the top layers and line wings. Fix: a Taylor expansion to $O(u^3)$ below `eps=1e-3`, applied with the `double-where` (`safe=where(u<eps,1,u)`) so the discarded branch never divides by zero. This is the make-or-break detail; it is verified below.

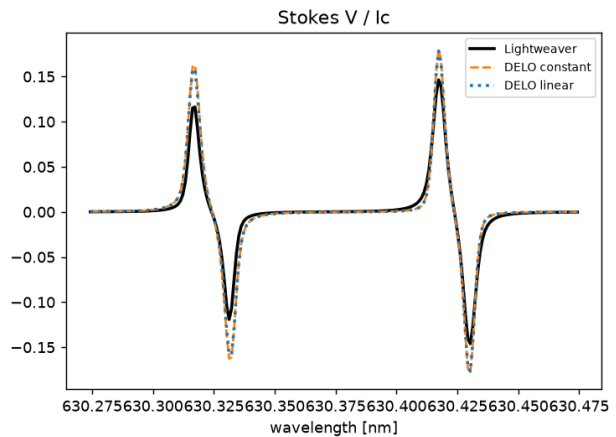
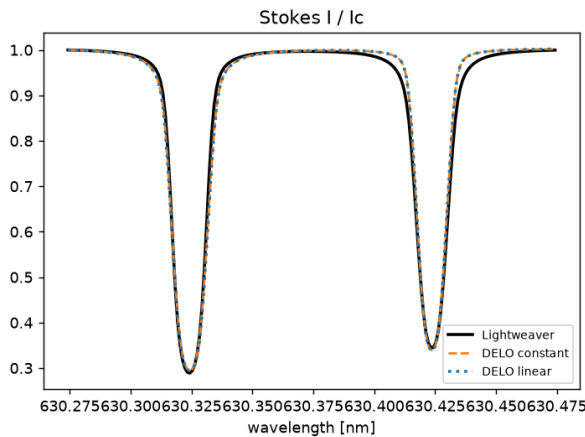
3. Validation (`formal_solvers/validate_linear.py`)

Gradients (clean). `jacrev(delo_linear_fs)` is finite even at $\Delta\tau = 8 \times 10^{-5}$ (guard works) and matches finite differences ($\sim 1e-8$ at thick layers; the lone `5.8e-4` is FD noise where the sensitivity is ≈ 0). Full-RT `jacrev` w.r.t. T is finite, $\max |\partial I / \partial T| = 2.15 \times 10^{-3}$ (matches the known f64 value).

Forward vs Lightweaver (FAL, fine grid). Linear $\approx$ constant (linear slightly better in Q, U, V ; tie in I); *both* sit $\sim 5\text{--}7\%$ from Lightweaver, a residual set by the opacity physics (Fe-only partition functions, simplified continuum), **not** the solver.

Coarse grid (isolates the solver: same opacity code, fine-grid linear = truth). This is the payoff — as layers thicken, linear clearly beats constant:

grid	linear	constant
skip=4 (21 layers)	3.6e-2	1.4e-1 ($\sim 4\times$ worse)
skip=8 (11 layers)	2.6e-1	5.2e-1 ($\sim 2\times$ worse)



FAL-C fine grid: Lightweaver vs DELO constant vs linear (normalised by own continuum); constant and linear overlie, and the offset from Lightweaver is opacity physics, not the solver.

4. Wired in, and a scalar twin

`delo_linear_fs` is now the solver used by `vector_response_fn.py` (`lte_polarised_rt`), so the whole synth/inversion path runs on it. A matching **scalar** solver `formal_solvers/scalar_formal_solver_linear.linear_fs` was added the same way (identical linear weights + small- $\Delta\tau$ guard) and wired into `response_fn.py` (`lte_rt`); it behaves the same — $\sim$ free on the fine grid, and an even larger coarse-grid gain, since the old single-point `nearest_fs` blows up ($\sim 2.4e4$ at skip=8) where linear stays ~ 0.3 . f64 throughout; files uncommitted.